

OBSERVACIONES DE LA PRÁCTICA

Sergio Andres Montejo Luna 202414786
Pablo Rocha 202520981

Ambientes de pruebas

	Máquina 1	Máquina 2	Máquina 3
Procesadores	AMD Ryzen 7 4700U with Radeon Graphics (2.00 GHz)	AMD Ryzen 7 7730U with Radeon Graphics (2.00 GHz)	
Memoria RAM (GB)	16,0 GB (15,4 GB usable)	16	
Sistema Operativo	Sistema operativo de 64 bits, procesador basado en x64	Windows 11	

Tabla 1. Especificaciones de las máquinas para ejecutar las pruebas de rendimiento.

Máquina 1

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.042	0.003	0.117
5.00%	0.591	0.002	1.106
10.00%	0.756	0.002	3.209
20.00%	1.332	0.005	9.794
30.00%	1.882	0.002	24.142
50.00%	3.402	0.003	57.538
80.00%	5.550	0.003	134.845
100.00%	7.250	0.003	211.221

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Linked List)
0.50%	0.022	0.002	0.105
5.00%	0.196	0.002	1.142
10.00%	0.640	0.003	3.119
20.00%	0.840	0.003	10.216
30.00%	1.379	0.002	20.783

50.00%	2.783	0.003	51.175
80.00%	3.471	0.003	128.674
100.00%	6.756	0.004	205.279

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.246	0.005	0.213
5.00%	0.560	0.002	1.065
10.00%	0.706	0.002	3,11
20.00%	1.518	0.003	12.735
30.00%	2.045	0.003	18.754
50.00%	3.236	0.002	47.643
80.00%	6.764	0.003	137.859
100.00%	7.465	0.002	210.470

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.044	0.044	0.139
5.00%	0.196	0.002	1.102
10.00%	0.443	0.002	4.134
20.00%	1.240	0.004	11.695
30.00%	4.492	0.003	21.028
50.00%	2.311	0.002	47.190
80.00%	3.958	0.004	124.322
100.00%	5.278	0.003	206.185

Máquina 2

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.084	0.094	0.004
5.00%	0.303	1.097	0.003
10.00%	0.764	3.298	0.002
20.00%	1.258	9.164	0.001
30.00%	1.613	20.131	0.001
50.00%	2.755	53.141	0.001

80.00%	4.536	151.835	0.002
100.00%	5.863	226.225	0.002

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%	0.03	0.077	0.005
5.00%	0.155	1.152	0.002
10.00%	0.311	3.484	0.002
20.00%	0.709	9.953	0.002
30.00%	1.021	21.054	0.002
50.00%	1.787	53.419	0.004
80.00%	3.209	137.382	0.002
100.00%	3.559	227.245	0.002

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.066	0.058	0.002
5.00%	0.379	1.124	0.002
10.00%	0.701	4.21	0.002
20.00%	1.11	9.98	0.002
30.00%	1.479	19.817	0.001
50.00%	2.809	52.381	0.002
80.00%	4.021	140.553	0.006
100.00%	5.07	222.356	0.001

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.017	0.052	0.001
5.00%	0.16	1.045	0.002
10.00%	0.494	3.806	0.002
20.00%	0.669	9.753	0.001
30.00%	1.027	20.098	0.002
50.00%	1.804	53.573	0.002
80.00%	3.178	139.537	0.002

Preguntas de análisis

- ¿Se observan diferencias significativas entre las implementaciones con ArrayList y LinkedList para las funciones de Queue y Stack? ¿Cuál es más eficiente en cada operación? ¿Por qué una implementación es más rápida en ciertos casos?
 - Si bien hay diferencias en la mayoría de las implementaciones, estas parecen ser aisladas (outliers). La única función en la que parece haber una diferencia marcada es en Enqueue; donde el uso de un array list es más eficiente a medida que el número de datos incrementa. Esto podría apuntar a que el uso de arraylist puede ser más eficiente para las colas ya que nos permite usar los índices para acceder rápidamente a los elementos con base a la posición que tienen dentro de la lista nativa de Python.
- ¿Cuándo es preferible usar ArrayList o LinkedList? Si insertamos y eliminamos con frecuencia, ¿qué estructura conviene más? Si accedemos aleatoriamente a elementos, ¿cuál es más eficiente?
 - Si la idea es insertar y eliminar con frecuencia la mejor opción sería usar una linked list, ya que con un arraylist es más probable que debamos recorrer toda la lista o incluso reasignar todos los datos a otro espacio en la memoria en caso de que se llene el espacio previamente asignado. Si la idea es acceder aleatoriamente a elementos lo mejor sería usar un arraylist, pues el uso de índices permite acceder al elemento en una posición específica sin tener que recorrer todo el arreglo, a diferencia del linked list.
- Durante la ejecución de las pruebas ¿Se presentan anomalías en los tiempos de ejecución que no se explican con la teoría?
 - Efectivamente, en las pruebas de la máquina 2 se muestra un pico inusual en la ejecución de peek para la linked list cuando se realizó con el 80% de los datos. Este pico no se puede explicar por el número de datos ya que el tiempo de ejecución es menor tanto para 50% como para el 100%.
- Complete la siguiente tabla de acuerdo con qué operación es más eficiente en cada implementación (marque con una x la que es más eficiente). Adicionalmente, explique si este comportamiento es acorde con lo enunciado en la teoría. Justifique las respuestas.

		Array List	Linked List	Justificación
QUEUE	Enqueue()		X	Esto si está acorde a la teoría, ya que solamente debe reasignar un par de punteros a dos nodos (el nuevo head, y el anterior head), a diferencia del array list en donde debería desplazar todos los elementos previos.
	Dequeue()	X		También está de acuerdo a la teoría, ya que al tener índices se puede acceder aún más rápido al último elemento de la fila.
	Peek()		X	Realmente la diferencia de tiempos no es tan grande en esta función. Esto sucede porque en ambas se tiene rápido acceso al elemento que se retorna en peek. Tanto por índice como por el puntero Head (first)
STACK	Push()	X		Esto no está de acuerdo a la teoría que vimos en clase.
	Pop()		X	En la práctica resulta ser más eficiente el uso de Linked list para pop. Quizá esto ocurre por el

				puntero que tiene el último nodo, permitiendo un rápido acceso a las últimas posiciones de la pila.
	Top()	X	X	El tiempo de ejecución es prácticamente el mismo para ambos.